FitControl

FitControl is a microservice system that docker compose up brings up in full: four application services, three PostgreSQL databases and a RabbitMQ broker. The frontend is not part of the compose setup and is treated as an external client. This document describes the target architecture: the gateway, meal planning and analytics services are not implemented yet, and docker-compose.yml is a reference definition of the infrastructure rather than a ready-to-run setup.

Infrastructure

The whole system is described by a single docker-compose.yml file in the repository root and runs on the developer's machine.

Parameter	Value
Host	developer machine with Docker (demo version)
Compose project	fitcontrol, 8 containers
Database images	postgres:16-alpine
Broker image	rabbitmq:4.3-management
AuthService images	.NET 10: sdk for build, aspnet for runtime
Restart policy	restart: unless-stopped for all containers

Technology stack

The stack is built on ASP.NET and PostgreSQL; Python is used only in the analytics service, and YARP and RabbitMQ connect the services.

a) .NET services: ASP.NET (.NET 10 LTS), EF Core 10 + Npgsql 10 (Npgsql.EntityFrameworkCore.PostgreSQL 10.0.x), EF migrations – AuthService; the meal planning service is also ASP.NET.
b) Python service: Python 3.14, connects to PostgreSQL via `DATABASE_URL` and to the broker via `RABBITMQ_URL` – analytics service.
c) API Gateway: YARP 2.3 (Yarp.ReverseProxy, reverse proxy on ASP.NET) – single entry point, prefix-based routing.
d) Authentication: JWT, RSA signature (RS256, key of at least 2048 bits), key identifier kid – AuthService issues tokens, the other services validate them
e) Databases: PostgreSQL 16 – three independent databases, one per service.
f) Message broker: RabbitMQ 4.3 with the management plugin – asynchronous events between services.
g) Containerization: Docker, Docker Compose – building and running the whole system.
h) Configuration: environment variables, root `.env`, `env_file`, user-secrets – passwords, connection strings, JWT parameters.
i) Frontend: not decided yet; external client, not part of compose.

Databases

PostgreSQL 16: three independent databases in three separate containers, one per service.

Additional running services

RabbitMQ 4.3 with the management plugin — broker for asynchronous events between services.

Scheduled services / commands

None: there are no separate cron jobs or schedulers in the system.

Additional on-demand internal services

None: all services run permanently, there are no services started on demand.

Other services / APIs

None: the system does not use any third-party APIs (mail, payment gateways, etc.).

DevOps

Deployment — manual, with docker compose up -d --build on the developer's machine. (demo version)

Projects

The system consists of four application services and four infrastructure containers.

Gateway – gateway:

a) Single entry point: routes requests to the services, validates tokens, applies rate limiting and CORS, no business logic.
b) Technology: YARP 2.3 on ASP.NET (.NET 10).
c) Repository path: ./gateway

Auth Service – auth-service:

a) Issues JWTs and stores users.
b) Technology: ASP.NET (.NET 10).
c) Repository path: ./services/auth-service (Dockerfile — AuthService/Dockerfile). Database: auth_db.

Meal Planning API – meal-planning-api:

a) Meal planning, exchanges events through RabbitMQ.
b) Technology: ASP.NET.
c) Repository path: ./services/meal-planning. Database: meal_planning.

Analytics Service – analytics-service:

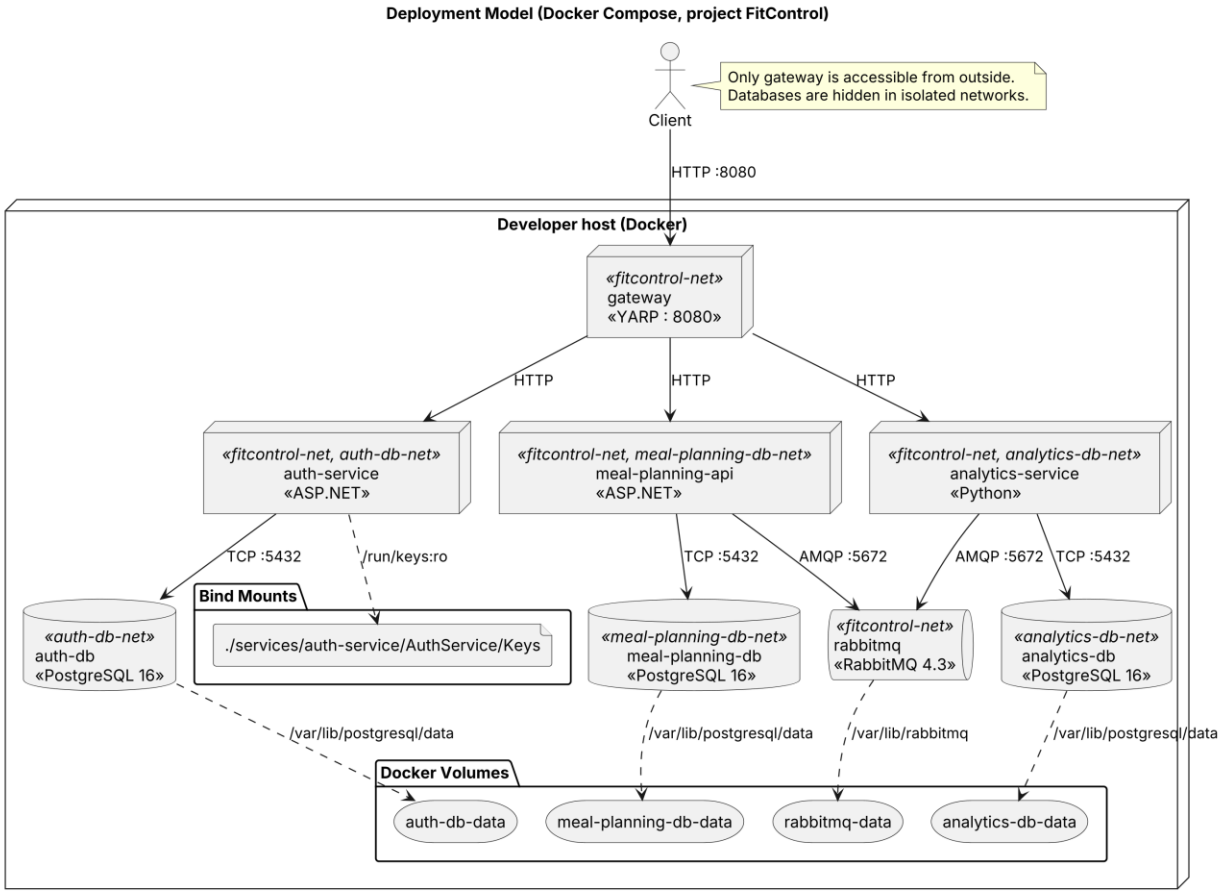
a) Analytics, receives events from RabbitMQ.
b) Technology: Python 3.14.
c) Repository path: ./services/analytics-service. Database: analytics_db.

Infrastructure containers

a) rabbitmq — broker for asynchronous events.
b) auth-db, meal-planning-db, analytics-db — data stores of their services.

Deployment model

In the target setup the client reaches only the gateway, the services exchange events with RabbitMQ inside the network, and each database is accessible only to its own service; the ports published on the host for development are listed in the Ports section. The development and demo environments are identical; only the host differs.



Communication model

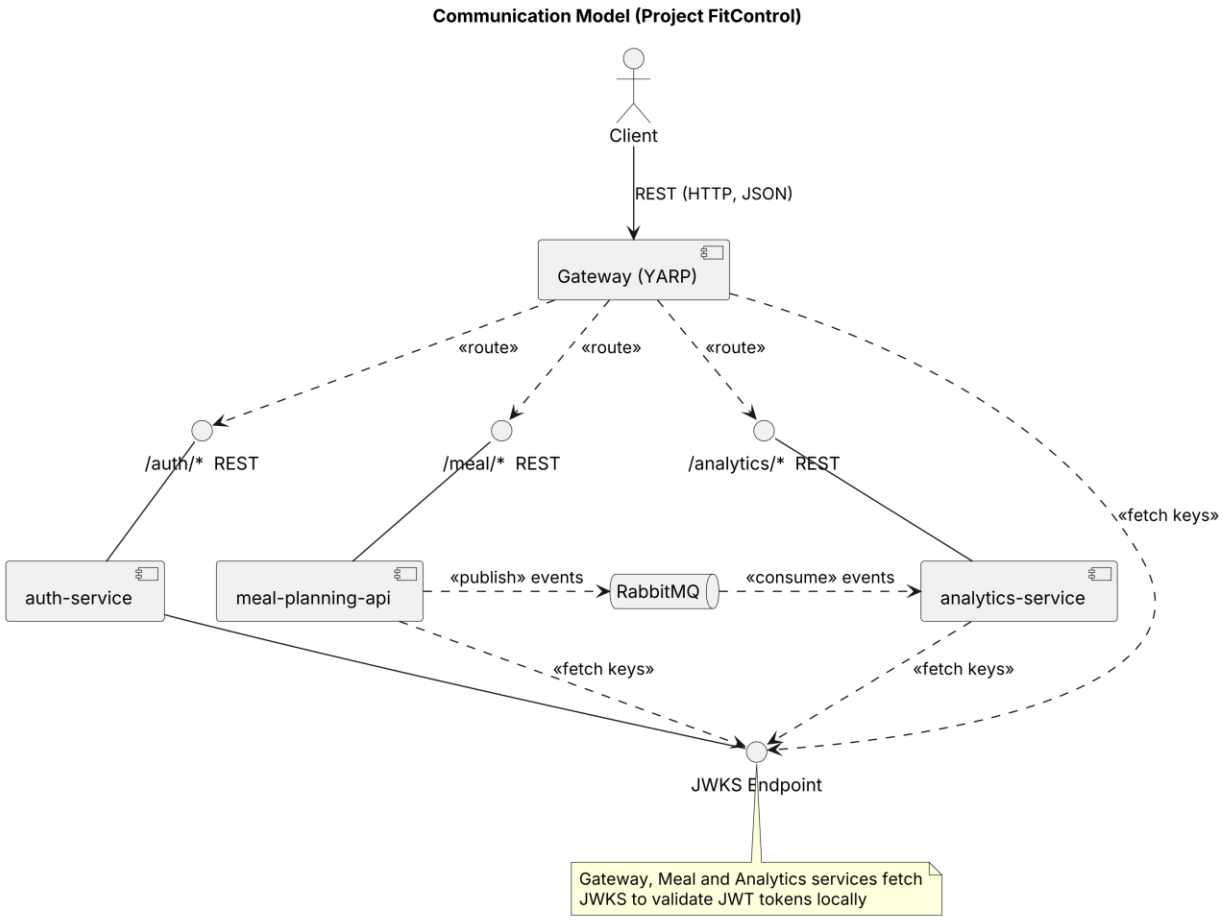
The target for communication between modules:

a) REST (HTTP, JSON) – every external call from the client, only through the gateway.

b) REST inside the network – the gateway calls services by name (for example, http://auth-service:8080) and routes requests by the prefixes /auth/*, /meal/*, /analytics/*.

c) AMQP via RabbitMQ – all asynchronous events between services (the publisher and consumer roles shown in the diagram are the target and are to be confirmed, see Open questions).

d) JWKS – the Gateway, Meal Planning and Analytics fetch the public key from AuthService and validate JWTs locally (to be confirmed, see Open questions).



Databases structure

The system uses three PostgreSQL 16 databases in three separate containers, each visible only to its own service. The database-per-service principle is enforced physically: three containers, three volumes, three networks.

Database rules:

a) the database name, user and password are set by variables with the service prefix: AUTH_DB_*, MEALPLANNING_DB_*, ANALYTICS_DB_*;
b) inside the network a service connects to its database by container name and port 5432;
c) the schema of a database is created by its service's migrations, not by a shared SQL script.

List of databases:

a) **auth_db** (container auth-db, user auth_user, volume auth-db-data, host port 5433 for development only)
   - owner: auth-service
   - the schema is created by EF Core migrations, the first one is InitialCreate
   - essential tables: Users (Id uuid, Email, PasswordHash, Role,CreatedAtUtc timestamptz) and the service table __EFMigrationsHistory
b) **meal_planning** (container meal-planning-db, user meal_planning, volume meal-planning-db-data, not published to the host)
   - owner: meal-planning-api
   - the schema is not defined yet, it will come with the service code
c) **analytics_db** (container analytics-db, user analytics_user, volume analytics-db-data, not published to the host)
   - owner: analytics-service

- the schema is not defined yet, it will come with the service code

Web access to the services

Address	What is there
http://localhost:8080	gateway — entry point to the system
http://localhost:5001	auth-service directly, bypassing the gateway
http://localhost:5002	meal-planning-api directly, bypassing the gateway
http://localhost:8000	analytics-service directly, bypassing the gateway
http://localhost:15672	RabbitMQ management UI
localhost:5433	auth_db database for pgAdmin and dotnet ef

Codebase structure

One monorepo: docker-compose.yml and .env.example in the root, the gateway in ./gateway and each service in its own folder under ./services, each with its own Dockerfile.

The AuthService Dockerfile is multi-stage:

a) base — the aspnet runtime image, the starting point of the final stage;
b) build — restores NuGet packages and builds the project;
c) publish — publishes the output files;
d) final — an aspnet image with the runtime only; the process does not run as root.

The paths ./services/auth-service, ./services/meal-planning, ./services/analytics-service and ./gateway are the agreed monorepo layout.

Developer's environment configuration

1. Install Docker and .NET SDK 10 (the SDK is needed only for migrations, the dotnet ef tool).
2. Copy .env.example to .env and replace every change_me.
3. Prepare the keys: the private_key.pem and public_key.pem pair appears in services/auth-service/AuthService/Keys the first time AuthService is run from the host in Development mode (key generation is enabled there), or it is placed there manually.
4. Check the file and start the system: docker compose config, then docker compose up -d --build.
5. Apply migrations from the host: set a connection string with Host=localhost;Port=5433 in user-secrets and run dotnet ef database update in the AuthService project folder. The container does not apply migrations itself.
6. Check the result: docker compose ps (databases are healthy), docker compose logs -f auth-service, the RabbitMQ UI at localhost:15672, the auth_db database in pgAdmin at localhost:5433.

Configuration: .env and signing keys

Secrets are kept outside the repository: passwords in the root .env, the private signing key in a folder mounted into the container as read-only.

1. Root .env. Compose reads it automatically and substitutes values into ${...} in docker-compose.yml. Only .env.example with change_me placeholders is stored in git.
2. What does not reach the container by itself. A value from .env reaches a service only if it is passed explicitly in the environment: block, for example Jwt__Issuer: ${JWT_ISSUER}.
3. Shared values (JWT_ISSUER, JWT_AUDIENCE, JWKS_URL, RABBITMQ_USER, RABBITMQ_PASSWORD) must match across all services, so they are defined once. All others carry the service prefix: AUTH_, MEALPLANNING_, ANALYTICS_.
4. Required passwords. Database and RabbitMQ passwords use the ${VAR:?message} form: without a value Compose will not start and will say what is missing.
5. Connection strings are assembled by Compose. AuthService receives ConnectionStrings__DefaultConnection with host auth-db and port 5432; when running from the host (dotnet run, dotnet ef) the user-secrets string with Host=localhost;Port=5433 is used.
6. Signing keys. AuthService reads private_key.pem and public_key.pem from /run/keys (RsaKeys__KeysPath). In Production mode key generation is disabled, so both files are required.
7. Restriction. Passwords must not contain ;, =, ' or $: they break connection strings.

The private key and the real .env never get into git or into an image: they are listed in .gitignore and .dockerignore.

Configure a new server

A "server" here means any machine where the system has to be brought up from scratch: a new teammate's laptop or a demo machine. Version compatibility matters: Docker Compose v2 (the docker compose command, not docker-compose) and .NET SDK 10 — the same version as in the AuthService images.

1. Install Docker Engine with the Compose v2 plugin (or Docker Desktop).
2. Clone the repository.
3. Create .env from .env.example and set the database passwords, RabbitMQ credentials and JWT parameters.
4. Put private_key.pem and public_key.pem into services/auth-service/AuthService/Keys: in Production mode AuthService does not generate keys and will not start without them.
5. Make sure ports 8080, 5001, 5002, 8000, 5433 and 15672 are free on the host, in particular that a locally installed PostgreSQL does not get in the way.
6. Start the system: docker compose up -d --build. Compose handles the start order itself: first the databases and RabbitMQ (until healthy), then the services, then the gateway.
7. Apply AuthService migrations from the host: dotnet ef database update with the connection string Host=localhost;Port=5433.

Re-deploying the application

1. Update one service without stopping the rest: docker compose up -d --build <service>. Data volumes are kept.

2. Reset the databases to a clean state: docker compose down -v — removes the volumes together with the data.

Releasing a new version

There is no formal release process yet: the system runs from the current state of the repository on the developer's machine. An image registry and CI will come with the move to orchestration (see "What would change in production").

Networks

Isolation is provided by four Docker networks: each database sits in its own network together with only its own service.

Network	Containers
fitcontrol-net	gateway, auth-service, meal-planning-api, analytics-service, rabbitmq
auth-db-net	auth-service, auth-db
meal-planning-db-net	meal-planning-api, meal-planning-db
analytics-db-net	analytics-service, analytics-db

Inside a network, containers reach each other by service name and internal port (for example, auth-db:5432), not via localhost and not via the host port.

Ports

Only one application port is open to the client – the gateway on 8080. The other ports are published to the host for development and debugging only and must be closed once the gateway is fully working.

Container	Container port	Host port	Variable	Why it is open
gateway	8080	8080	GATEWAY_PORT	entry to the system
auth-service	8080	5001	AUTH_PORT	debugging without the gateway
meal-planning-api	8080	5002	MEALPLANNING_API_PORT	debugging without the gateway
analytics-service	8000	8000	ANALYTICS_PORT	debugging without the gateway
auth-db	5432	5433	AUTH_DB_PORT	pgAdmin and dotnet ef from the host

Container	Container port	Host port	Variable	Why it is open
rabbitmq	15672 (management UI)	15672	—	viewing queues; AMQP port 5672 is not exposed

The meal-planning-db and analytics-db databases are not published to the host.

Volumes and mounts

a) Docker volumes: auth-db-data, meal-planning-db-data, analytics-db-data – one per database, rabbitmq-data – for the broker. Data survives container re-creation; docker compose down -v removes it.
b) Bind mount: the ./services/auth-service/AuthService/Keys folder is mounted into auth-service as /run/keys, read-only – it holds the JWT signing keys.

Communication channels

Channel	Protocol	Participants	Status
Client → gateway	HTTP, JSON	frontend and gateway	the only application port open to the outside

Channel	Protocol	Participants	Status
Gateway → services	HTTP (REST)	gateway and the three services	routing by prefix
Events	AMQP (internal port 5672)	meal-planning-api, analytics-service, rabbitmq	connection configured in compose, the set of events is not defined yet
JWT validation	HTTP, key request via JWKS_URL	gateway, meal-planning-api, analytics-service fetch the public key from auth-service	needs agreement (see "Open questions")

Communication rules

a) A service never accesses another service's database. Data of different services is linked only by the user identifier from the token (claim sub); there are no foreign keys between databases.

b) Services do not call each other in chains: the client reaches each service through the gateway, and dependencies between services are passed as events (the only exception is the cached fetch of public keys from AuthService, see JWKS above).

c) The gateway stays a thin proxy: routing, token validation, rate limiting and CORS, no business logic.

d) The broker is used only inside the network; management port 15672 is open to the host for development only.

Known issues

Symptom	Cause	What to do
PostgreSQL error 28P01, password rejected	the volume keeps the password set when it was first created; or another PostgreSQL installed on the host answers on port 5432	docker compose down -v and start again; connect to the container on port 5433
Migration fails with "relation Users already exists"	one database was cleaned, but the migration is applied to another one (usually the server on the host)	check host and port in the connection string
Build fails with Invalid reference: "	Source Link tries to read git data inside the container	add -p:EnableSourceControlManagerQueries=false to dotnet build and dotnet publish

Symptom	Cause	What to do
AuthService does not start, cannot find the key	public_key.pem is missing or the folder is not mounted at /run/keys	put both files into services/auth-service/AuthService/Keys
A service cannot connect to its database	the connection string says localhost, which inside the container is the container itself	use the service name, for example Host=auth-db

Open questions

Six questions remain open; the main one is how services validate JWTs.

a) JWT validation. Teammates' services expect a shared secret (Jwt__Key, JWT_SECRET), while AuthService signs tokens with RSA and already publishes its public key at /.well-known/jwks.json. The target is validation via JWKS (JWKS_URL) in all services; teammates need to switch from the shared secret.

b) Variable names. Compose uses Jwt__JwksUrl for .NET and JWKS_URL for Python; teammates should confirm them or name their own.

c) RabbitMQ events. It is not yet defined who publishes and who consumes events, what they are called and in what format (JSON). For the first version one "one publisher, one consumer" scenario is enough.

d) Service name. In the teammate's code it is called meal-planning, in the project plan — the planner; one name should be chosen for the container, database and documentation.

e) Healthcheck for .NET services. The aspnet image has no curl: either install it in the Dockerfile or do not use the healthcheck in depends_on.

f) Gateway project. The YARP project has not been created yet.

What would change in production

The setup is designed for demos and local development; production would require seven changes.

a) Secrets. A secret manager (Docker secrets, Azure Key Vault or similar) instead of .env and mounted keys; keep the private signing key there and rotate it by the kid identifier.

b) Data. Move databases to separate servers or managed PostgreSQL, set up backups, do not publish database ports to the host. Store AuthService's DataProtection keys in a volume or external storage: right now they disappear together with the container.

c) TLS. Terminate TLS at the gateway or at a proxy in front of it; inside the network services talk over HTTP.

d) Broker. Add retries, dead-letter queues and the outbox pattern so events are not lost when a service fails.

e) Observability. Centralized logs, an end-to-end correlation ID and distributed tracing; real /health endpoints in every service.

f) Orchestration. Move to Kubernetes: named volumes become PersistentVolumeClaims, databases run as StatefulSets.

g) CI/CD and registry. Build images in a CI pipeline, push them to an image registry and deploy by version tag instead of building on the developer's machine.